

MODULE-6:

Here are the important points in the provided document:

AWS Academy Cloud Architecting Module 6: Adding a Database Layer

This document is the student guide for the AWS Academy Cloud Architecting course, specifically Module 6: Adding a Database Layer.

Module Objectives

- Compare different types of databases and the services AWS offers.
- Understand when to utilize Amazon RDS.
- Explain the advanced capabilities of Amazon RDS.
- Know when to employ Amazon DynamoDB.
- Identify the purpose-built database services offered by AWS.
- Explain the process of migrating data to AWS databases.
- Learn to design and deploy a database server.
- Apply AWS Well-Architected Framework principles to the design of a database layer.

Module Structure

The module includes the following components:

- **Presentation Sections:** Covering various topics such as database layer considerations, Amazon RDS, Amazon RDS Proxy connection management, Amazon DynamoDB, purpose-built databases, data migration into AWS databases, and applying AWS Well-Architected Framework principles.
- **Demo:** Focusing on Amazon RDS Automated Backup and Read Replicas.
- **Knowledge Checks:** A 10-question assessment and a sample exam question for class discussion.
- **Hands-on Labs:** A guided lab for creating an Amazon RDS database and a challenge lab on migrating a database to Amazon RDS.

Database Layer Considerations

- **Scalability:** Choose a database that can handle current throughput and scale as needed.
- **Storage Requirements:** Consider the size of the database and its storage needs (gigabytes, terabytes, or petabytes).
- **Data Characteristics:** Understand the data model, access patterns, latency requirements, and record size.
- **Durability:** Determine the level of durability, availability, and recoverability needed and consider data residency or regulatory obligations.

Relational vs. Non-Relational Databases

- **Relational Databases:** Use a tabular format (columns and rows), adhere to strict schema rules, excel in data integrity, commonly use SQL, are suitable for migrating on-premises relational workloads or online transactional processing (OLTP).
 - Relational database transactions are **ACID-compliant**, meaning they are atomic, consistent, isolated, and durable.
- **Non-Relational Databases (NoSQL):** Designed for specific data models (key-value, graph, document, in-memory, search), handle structured, semi-structured, and unstructured data, employ flexible schemas, excel in flexibility, scalability, and high performance, are suitable for caching layers, JSON document storage, or low-latency data retrieval.
 - Both relational and non-relational databases can scale both horizontally and vertically.

AWS Database Options

- **Relational:** Amazon RDS is the primary option, a managed service offering seven familiar database engines.
- **Non-Relational:** Includes purpose-built options like DynamoDB (key-value), Neptune (graph), and ElastiCache (in-memory).

Benefits of AWS Managed Database Services

Reduce responsibilities for tasks like power management, server maintenance, OS installation and patching, database installation and patching, backups, high availability, and scaling.

Database Capacity Planning

- Analyze current storage capacity.
- Predict future capacity needs.
- Determine if horizontal scaling (adding servers) or vertical scaling (increasing server resources) is necessary.

Amazon RDS

- **Managed service:** Simplifies setup, operation, and scaling of relational databases.
- **Supports multiple database engines:** Aurora with MySQL compatibility, Aurora with PostgreSQL compatibility, RDS for MySQL, RDS for MariaDB, RDS for PostgreSQL, RDS for Oracle, and RDS for SQL Server.
- **Uses Amazon EBS for storage:** Database and log storage utilize EBS volumes.

Benefits of Amazon RDS

- **Reduced Administrative Burden:** Eliminates the need to provision infrastructure or manage database software.

- **High Scalability:** Easily scale compute and storage resources.
- **Availability and Durability:** Options for automated backups, snapshots, host replacement, and Multi-AZ deployments for high availability and failover.
- **Security and Compliance:** Isolation in VPCs, firewall settings, encryption at rest and in transit, and compliance readiness (e.g., HIPAA).

Amazon Aurora

- **Cloud-built RDBMS:** Offers MySQL and PostgreSQL compatibility.
- **Managed by Amazon RDS:** Provides the benefits of a managed service.
- **High Performance and Availability:** Delivers up to five times faster performance for MySQL and three times faster for PostgreSQL compared to standard databases.
- **Cost-Effective:** Achieves performance at one-tenth the cost of commercial databases.
- **Scalable and Fault-Tolerant:** Scales up to 64 TB per instance and offers high availability with up to 15 read replicas, point-in-time recovery, and replication across three Availability Zones.

Aurora Database Clusters

- **Cluster Volume:** A virtual storage volume spanning multiple Availability Zones, ensuring data redundancy.
- **Primary Instance:** Handles read and write operations.
- **Aurora Replicas:** Support read operations, offload read workloads, and provide automatic failover.

Aurora Serverless

- **On-Demand Auto Scaling:** Automatically adjusts capacity based on application needs.
- **Hands-Off Capacity Management:** Simplifies database management.
- **Fine-Grained Scaling:** Matches capacity closely to demand.
- **Suitable for:** Variable workloads, new applications, development and testing, and capacity planning.

Amazon RDS Use Case: Banking Transactions

Demonstrates using Amazon RDS for storing and managing high-volume transactional data with ACID compliance, such as banking transactions.

Amazon RDS Instance Types and Sizing

- **General Purpose:** Suitable for CPU-intensive or moderate CPU workloads with temporary spikes (T4g, T3, M6g, M5).
- **Memory-Optimized:** Designed for query-intensive workloads or high connection counts (R6g, R5, X2g, X1E).

Amazon RDS Security Best Practices

- **VPC Deployment:** Use a VPC for network access control.
- **IAM Policies:** Control access to RDS resources.
- **Security Groups:** Manage connections to databases.
- **SSL/TLS Connections:** Encrypt data in transit for supported engines.
- **Encryption at Rest:** Use AWS KMS keys to encrypt database instances and snapshots.
- **Database Engine Security:** Leverage engine-specific security features.

Amazon RDS Proxy

- **Fully Managed Proxy:** Improves scalability, resilience, and security.
- **Connection Pooling:** Pools database connections for better application scaling.
- **Seamless Failover:** Reduces failover times and preserves application connections.
- **Streamlined Authentication:** Enforces IAM authentication and stores credentials in Secrets Manager.

Backing Up Data in Amazon RDS

- **Automated Backups:** Daily snapshots and transaction log capture.
- **Database Snapshots:** User-initiated backups for point-in-time recovery.
- **Cross-Region Backups:** Replicate snapshots and transaction logs to another Region for disaster recovery.
- **Encryption for Backups:** Protect backup data with encryption using AWS KMS.

Amazon DynamoDB

- **Fully Managed NoSQL Database:** Supports key-value and document data models.
- **Serverless and Scalable:** Delivers millisecond performance and auto-scales tables.
- **Use Cases:** Application development, mission-critical workloads requiring speed, scalability, and data durability.

DynamoDB Use Cases

- **Software Application Development:** Handling user content metadata, caches, high concurrency, and large-scale applications.
- **Media Metadata Stores:** Scaling for real-time video streaming and interactive content.
- **Gaming Platforms:** Managing player data, session history, and leaderboards for millions of users.

DynamoDB Features

- **Serverless Performance and Scalability:** Includes secondary indexes, DynamoDB Streams for event-driven architectures, and global tables for multi-Region replication.
- **Built-in Security and Reliability:** Offers encryption at rest, point-in-time recovery, and fine-grained access control.

DynamoDB Data Structure

- **Tables:** Collections of data items.
- **Items:** Groups of attributes uniquely identified by a primary key.
- **Attributes:** Fundamental data elements in key-value pairs.
- **Primary Key:** Consists of a partition key (hash key) and an optional sort key.
- **Secondary Indexes:** Allow querying data based on attributes other than the primary key.

Global Secondary Index (GSI)

- **Alternate Schema:** Creates a read-only copy of the base table with different partition and sort keys.
- **Eventual Consistency:** Data consistency is eventually achieved, typically within one second.

Local Secondary Index (LSI)

- **Strong Consistency:** Provides strongly consistent reads for specific query patterns.
- **Limited Flexibility:** Partition key must be the same as the base table.

DynamoDB Global Tables

- **Multi-Region Replication:** Replicates tables across multiple AWS Regions for fast local read and write performance.
- **High Availability:** Ensures data accessibility even during regional outages.

DynamoDB Security Best Practices

- **Preventative:** IAM roles, policies, conditions for fine-grained access control, and VPC endpoints.
- **Detective:** CloudTrail monitoring, AWS Config monitoring and compliance rules.

Purpose-Built Databases

- **Amazon Redshift:** A data warehousing service for petabyte-scale analytics workloads, offering columnar storage and a serverless option.
- **Amazon DocumentDB:** A MongoDB-compatible document database for flexible schemas and complex document storage.
- **Amazon Keyspaces:** A Cassandra-compatible database service for high-volume data querying and scalability.
- **Amazon MemoryDB:** A Redis-compatible in-memory database service for ultra-fast performance and low latency.
- **Amazon Neptune:** A graph database service for navigating highly connected datasets and finding data relationships.
- **Amazon Timestream:** A timeseries database service for analyzing trends and patterns in time-sequenced data.
- **Amazon QLDB:** A ledger database for maintaining an immutable and verifiable history of data changes.

Matching a Database to Your Business Needs

- **Suitable Workloads:** Consider if the database capabilities align with the workload requirements.
- **Data Model:** Understand the characteristics of the data model.
- **Features and Benefits:** Utilize key features and configuration options for optimal performance.
- **Common Use Cases:** Review relevant use cases for reference architectures and examples.

Migrating Data into AWS Databases

AWS Database Migration Service (AWS DMS)

- **Managed Service:** Facilitates data migration and replication.
- **Supports Homogenous and Heterogenous Migrations:** Migrate databases with the same or different engines.
- **Wide Database Support:** Compatible with various commercial and open source databases.
- **Continuous Replication:** Replicates data on demand or on schedule.

AWS DMS Homogenous Migration

- **Simplified Migration:** Streamlines migration between the same database engines (e.g., on-premises PostgreSQL to Amazon RDS for PostgreSQL).
- **Serverless and Automated:** AWS DMS handles resource scaling and uses native database tools.

Tools for Heterogenous Database Migrations

- **AWS DMS Fleet Advisor:** Discovers and assesses on-premises databases, recommends migration paths.
- **AWS Schema Conversion Tool (AWS SCT):** Converts source schemas and code to target formats.
- **AWS DMS Schema Conversion:** A centrally managed service for schema assessment and conversion within AWS DMS workflows.

AWS Well-Architected Framework Principles for the Database Layer

- **Performance Efficiency:** Emphasize choosing appropriate database architectures based on data and access patterns, using read replicas, global tables, partitioning, and caching.
- **Security:** Focus on secure key management with AWS KMS and enforcing encryption at rest.
- **Cost Optimization:** Guide the selection of resource types, sizes, and numbers based on data characteristics and workload needs.

Café Lab Scenario

- Highlights the challenges of managing an on-premises database, such as manual backups and patching.
- Demonstrates migrating a database to Amazon RDS to reduce administrative burden and improve efficiency.

Conclusion

The module provides a comprehensive overview of database concepts, AWS database services, data migration tools, and the application of the AWS Well-Architected Framework principles. It aims to equip students with the knowledge and skills needed to design and deploy efficient, scalable, and secure database solutions in the AWS cloud.